

Chapter 11: Reinforcement Learning

Outline

Introduction

Fundamentals

Reinforcement Learning Algorithms

Applications and challenges

Conclusions

11.1 Introduction

- ▶ Predictive models, be it regression or classification, *describe* the state of the system.
- ▶ With these descriptions we can then make informed decisions and determine the best course of action.
- ▶ Can we train a computer so that it learns *how to react* to the current state of the system?
- ▶ Reinforcement learning (RL) focuses on this very question.

11.1 Introduction

- ▶ RL is a subfield of AI focus on building computational *agents* that can *perceive* their environment and *act* so that a goal or objective is achieved.

11.2 RL Fundamentals - Data

- ▶ For an RL agent to learn how to react to its environment, it must have a way of perceive it and interact with it.
- ▶ In game of chess, the environment is the board and the position of the pieces (perceived by the agent as a matrix), and the way to interact with is by moving the pieces following the rules of the game.
- ▶ In engineering applications, the environment is typically described by the state variables (pressure, temperature, etc.) of the process, and we interact with it through valves and other actuators.

11.2 RL Fundamentals - Data

- ▶ Since safety and cost limitations prevent us from giving the computer access to a real plant to learn, we must rely on simulations.
- ▶ This means that a predictive model becomes the environment for the RL agent and all the consideration that discussed in the previous chapters apply.
- ▶ If we allow the agent to perceive the environment through, say, images, the ideas from computer vision become relevant.

11.2 RL Fundamentals - Components

- ▶ An environment holds the dynamics of the task being solved by the RL agent.
- ▶ Relevant information about the status of the environment is held within the *state* (s_t).
- ▶ An agent uses this information to choose an *action* (a_t)
- ▶ In response to the agent's action, the environment moves to a new state s_{t+1} and sends out a *reward* (r_{t+1}) signaling the quality of the action.

11.2 RL Fundamentals - Components

- ▶ The playbook that the agent uses to make its decisions is called *policy*.
- ▶ The agent keeps a record of its actions and their corresponding rewards for each state, and builds a map called *value function*.
- ▶ If there are preferred states or actions, the agent uses the value function to find learn a policy that favors the preferences.

11.2 RL Fundamentals - Solving an RL Problem

- ▶ The agent samples actions in the environment during an *episode* following a policy $\pi(a|s)$, which describes the probability of choosing a particular action a while being in a state s for each *step* in the episode.
- ▶ By following the policy, the agent generates a sequence of states, actions, and rewards.
- ▶ A greedy solution would be to take the action that produces the highest reward at every point, but it is likely that this policy will have low potential for future reward.

11.2 RL Fundamentals - Solving an RL Problem

- ▶ Much like optimization problems, RL problems are solved using algorithms.
- ▶ RL algorithms are set up to maximize the *sum of future rewards*, also known as *return*, R_t .

$$R_t = \sum_k \gamma^k r_{t+k} \quad (1)$$

- ▶ Here $\gamma \in [0, 1]$ is a discount factor that assigns more value to the most immediate rewards.

11.2 RL Fundamentals - Solving an RL Problem

- ▶ Value functions store the *expected* returns associated with each state (or combination of state and action).
- ▶ There are two types of value functions:
 - ▶ V^π , defined only over states
 - ▶ Q^π , defined over states and actions.
- ▶ The agent's goal is to learn a policy π^* that maximizes the expected return.
- ▶ The policy is updated after each episode by estimating the actions (and states) at each step that maximize the return according the current value function.
- ▶ Solving a RL problem using a Q^π value function is also called Q-learning, and is one of basic forms of RL.

11.2 RL Fundamentals - Q-learning Principle

Algorithm 1: Q-learning with ϵ -greedy policy

Input: Learning rate α , discount factor γ , exploration rate ϵ

Output: Action-value function $Q(s, a)$

Initialize $Q(s, a)$ arbitrarily for all states s and actions a ;

for *each episode* **do**

 Initialize state $s \leftarrow s_0$;

for *each step* **do**

 Choose action a :

$a \leftarrow \arg \max_a Q(s, a)$ with probability $1 - \epsilon$

$a \leftarrow \text{random}(a)$ with probability ϵ

 Take action a , observe reward r and next state s' ;

$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$;

$s \leftarrow s'$;

end

end

11.2 RL Fundamentals - Value functions

- ▶ So far we have mentioned the idea of value functions and how important they are in RL.
- ▶ But, how exactly does a value function look like?
- ▶ In its simplest form, a value function is a look-up table:

	State s	Value $V(s)$
$V(s) =$	s_1	0.0
	s_2	1.2
	s_3	-0.5
	s_4	2.3

- ▶ A value function can also be a predictive model, like the ones discussed in the previous chapters.

11.2.4 Deep Reinforcement Learning

- ▶ Much like value functions, policies and environments can be represented by models.
- ▶ As RL problems become more complex, the models used become more sophisticated.
- ▶ The branch of RL that uses ML models for value functions, policies and/or environments is known as Deep Reinforcement Learning (DRL).

11.3 RL Algorithms

- ▶ As mentioned before, RL problems are solved using RL algorithms.
- ▶ These algorithms fall into three categories:
 - ▶ Q-learning: focus on learning the value function, and calculate policy directly from value function.
 - ▶ Policy gradient: have separate model that estimates the policy, focus on learning this model.
 - ▶ Actor-critic: combine both Q-learning and policy gradient.

11.3 RL Algorithms - Policy Gradient

- ▶ Use an ANN to represent the policy.
- ▶ Train the policy network π_θ by minimizing the loss function:

$$L_{PG} = \hat{E}_t \left[\log \pi_\theta(a_t | s_t) \hat{A}_t \right] \quad (2)$$

- ▶ Here $\hat{A}_t$ is the estimator for an advantage function at time t , which is the difference between the value function and the Q function.
- ▶ Examples of policy gradient algorithms include:
 - ▶ Proximal policy gradient optimization (PPO)
 - ▶ Deep deterministic policy gradient (DDPG)

11.3 RL Algorithms - Actor-Critic

- ▶ Use a policy network $\pi(\theta_p)(s_t)$ that generates a probability distribution of favorable actions given the current state s_t .
- ▶ Use a value network $V_{\theta_v}(s_t)$ that evaluates the expected return from state s_t .
- ▶ Learn θ_p and θ_v simultaneously via training.
- ▶ The loss function for $V_{\theta_v}(s_t)$ is

$$L_V = E[R_t - V_{\theta_v}(s_t)] \quad (3)$$

- ▶ Examples of policy gradient algorithms include:
 - ▶ Asynchronous advantage actor-critic (A3C)
 - ▶ Twin delayed deep deterministic policy gradient (TD3)

11.4 DRL for Process Control

- ▶ Although DRL has achieved major success in games such as chess and video games, its ability to achieve intelligent control behavior in chemical reaction systems remains uncertain due to fundamental differences between these environments.
- ▶ Unlike games, real-time exploration in chemical plants is impractical because of time, material, and safety constraints; additionally, games provide clear observations, rewards, and rules, making them well suited for DRL optimization.

11.4 DRL for Process Control

- ▶ Chemical systems lack clearly defined control objectives, are highly sensitive to objective function design, rely on accurate and fast simulation models for optimization, and often exhibit significant time delays between actions and observable effects, complicating DRL training.
- ▶ Further challenges include the high computational cost of simulating complex reaction systems, the presence of many measured and unmeasured variables, and potential model inaccuracies, all of which increase the difficulty of applying DRL to chemical process control.

11.5 Conclusions

- ▶ This chapter presented an overview of RL in chemical engineering, situating it within machine learning and highlighting its role in decision-making for complex systems such as chemical reactions, with DRL extending these capabilities to more challenging optimization problems.
- ▶ The fundamental components of the RL framework were discussed, with particular emphasis on value functions and their role in guiding agents to maximize expected returns.
- ▶ While RL faces challenges in chemical process control, including safety, time delays, and computational efficiency, case studies demonstrate its potential and help clarify both the capabilities and limitations of RL in real-world chemical engineering applications.